

LangChain & Agentic AI – From Basics to Advanced demo-10 Notes

2 ways to provide default values:

1. By using Python's support:

```
topic = input("Enter Topic: ") or "Python"
```

```
level = input("Enter Level: ") or "Beginner"
```

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
prompt_template = PromptTemplate.from_template(
```

```
    "Explain {topic} at {level} level"
```

```
)
```

```
topic = input("Enter Topic: ") or "Python"
level = input("Enter Level: ") or "Beginner"
```

```
prompt_value = prompt_template.invoke(
    {
        "topic":topic,
        "level":level
    }
)
```

```
print(prompt_value.to_string()) #generated final prompt
print()
print()
```

```
llm = ChatOpenAI(
    model = "gpt-4o-mini",
    api_key = os.getenv("OPENAI_API_KEY")
```

```
)
```

```
#response = llm.invoke(prompt_value)
```

```
#print(response.content)
```

2. LangChain partial_variables concept to provide default values:

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
prompt_template = PromptTemplate.from_template(
```

```
    "Explain {topic} at {level} level",
```

```
    partial_variables =
```

```
{
```

```
    "topic": "Python",
```

```
        "level":"Beginner"
    }
)
```

```
prompt_value = prompt_template.invoke(
    {
        "topic":"Java",
        #"level":"Advanced"
    }
)
```

```
print(prompt_value.to_string()) #generated final prompt
print()
print()
```

```
llm = ChatOpenAI(
    model = "gpt-4o-mini",
    api_key = os.getenv("OPENAI_API_KEY")
```

)

```
#response = llm.invoke(prompt_value)
#print(response.content)
```

Without using PromptTemplate class:

1. By using String concatenation operator(+)

```
import os

from langchain_openai import ChatOpenAI

topic = input('Enter Topic:')
language = input('Enter Language:')
level = input('Enter Level:')

prompt = "Explain "+topic+" In "+language+" For a "+level+"
Level Learner"
```

```
print(prompt)
```

```
llm = ChatOpenAI(  
    model = "gpt-4o-mini",  
    api_key = os.getenv("OPENAI_API_KEY")  
)
```

```
#response = llm.invoke(prompt)
```

```
#print(response.content)
```

Some conclusions:

1. One PromptTemplate can contain multiple variables.

2. Variables are written inside curly braces.

Example:

```
{topic}
```

```
{language}
```

```
{level}
```

3. We provide values using a dictionary.

```
{  
    "topic": "Python",  
    "language": "English",  
    "level": "Beginner"  
}
```

4. Variable names must match.

5. `template.input_variables` helps us inspect required variables.

6. `PromptTemplate` builds the dynamic prompt.

7. The model generates the AI response.

8. Multiple variables make one application reusable for many user requirements.

One Template + Different Data = Different Prompts

Different Prompts + Same LLM = Different AI Responses